

Побудова моделі

Мета

Ознайомитись з різновидами регресійних моделей. Після завершення цієї лабораторної роботи ви зможете:

- Будувати регресійні моделі кількох видів
- Оцінювати якість моделі візуально
- Оцінювати якість моделі за допомогою числових мір, без використання тестової вибірки
- Виконувати прогнозування відгуку, використовуючи побудовану модель

Завдання, що оцінюються

1. Скачати дані з фінального файлу першої роботи з виправленими помилками та заповненими пропусками. Записати дані у dataframe. В попередній роботі ви визначили ознаки, що можуть бути предикторами для 'CO2 emission'. Побудуйте моделі лінійної регресії для кожного з цих предикторів.
2. Побудуйте модель множинної лінійної регресії для всіх доречних предикторів разом.
3. Побудуйте кілька поліноміальних моделей другого порядку.
4. Побудуйте візуалізації для оцінки всіх моделей.
5. Порахуйте значення R^2 та MSE для оцінки якості кожної моделі. Оберіть найкращу модель.

Завдання #1:

Зчитую дані з файлу у датафрейм та імпортую бібліотеки, потрібні для побудови моделей.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score

path = "data/clean_data.csv"
df = pd.read_csv(path)
df.head()
```

```
Out[1]:
```

	Country Name	Region	GDP per capita	Population	CO2 emission	Area	Density
0	Afghanistan	South Asia	561.778746	34656032.0	9809.225	652860.0	53.08340
1	Albania	Europe & Central Asia	4124.982390	2876101.0	5716.853	28750.0	100.03829
2	Algeria	Middle East & North Africa	3916.881571	40606052.0	145400.217	2381740.0	17.04890
3	American Samoa	East Asia & Pacific	11834.745230	55599.0	6501.591	200.0	277.99500
4	Andorra	Europe & Central Asia	36988.622030	77281.0	462.042	470.0	164.42766

З попередньої роботи та кореляційного аналізу знаємо, що найкращими кількісними показниками для прогнозування `CO2 emission` можуть бути:

- `Population`
- `Area`

Розробимо моделі, використовуючи ці ознаки.

```
In [2]: df[['Population', 'Area', 'GDP per capita', 'Density', 'CO2 emission']].corr()
```

```
Out[2]: CO2 emission    1.000000
Population    0.804091
Area          0.551697
GDP per capita 0.088426
Density       -0.023559
Name: CO2 emission, dtype: float64
```

Знаходжу коефіцієнти моделі

```
In [3]: lm1 = LinearRegression()  
X1 = df[['Population']]  
Y = df['CO2 emission']  
lm1.fit(X1, Y)  
  
print('intercept =', lm1.intercept_)  
print('coef =', lm1.coef_)
```

```
intercept = -8619.50501753707  
coef = [0.00484832]
```

Отримали кінцеву лінійну модель зі структурою:

$$\hat{Y}_{hat1} = a + bX$$

Підставляючи фактичні значення, маємо:

```
In [4]: # CO2 emission = -8619.5050 + 0.00484832 * Population
```

Створюю об'єкт лінійної регресії та навчаю другу модель

```
In [5]: lm2 = LinearRegression()  
X2 = df[['Area']]  
lm2.fit(X2, Y)  
lm2
```

```
Out[5]: ▼ LinearRegression ⓘ ?  
        ► Parameters  
        ► Fitted attributes
```

Знаходжу коефіцієнти моделі

```
In [6]: print('intercept =', lm2.intercept_)  
        print('coef =', lm2.coef_)
```

```
intercept = 3646.087523958151  
coef = [0.23136652]
```

Отримали кінцеву лінійну модель зі структурою:

$$\hat{Y}_{hat2} = a + bX$$

Підставляючи фактичні значення, маємо:

```
In [7]: # CO2 emission = 3646.0875 + 0.23136652 * Area
```

Завдання #2:

Побудуйте модель множинної лінійної регресії для всіх доречних предикторів разом.

Створюю об'єкт лінійної регресії та навчаю множинну лінійну модель із структурою

$$\hat{Y}_3 = a + b_1 X_1 + b_2 X_2$$

```
In [8]: lm3 = LinearRegression()  
X3 = df[['Population', 'Area']]  
lm3.fit(X3, Y)  
lm3
```

```
Out[8]:  
▼ LinearRegression ⓘ ?  
  ► Parameters  
  ► Fitted attributes
```

Знаходжу коефіцієнти моделі

```
In [9]: print('intercept =', lm3.intercept_)  
        print('coef =', lm3.coef_)
```

```
intercept = -57898.205938172  
coef = [0.00419112 0.10806919]
```

Підставляючи фактичні значення, маємо:

```
In [10... # CO2 emission = -57898.2059 + 0.00419112 * Population + 0.10806919 * Area
```

Завдання #3:

Побудуйте кілька поліноміальних моделей другого порядку.

Створюю об'єкт поліноміальної регресії та навчаю поліноміальну модель із структурою

$$\hat{Y}_4 = a + b_1 X_1 + b_2 X_1^2$$

Знаходжу коефіцієнти моделі

```
In [11... x1 = df['Population']
y = df['CO2 emission']

f4 = np.polyfit(x1, y, 2)
p4 = np.poly1d(f4)
print(p4)
```

$$-3.855\text{e-}13 x^2 + 0.005332 x - 1.777\text{e}+04$$

Створюю об'єкт поліноміальної регресії та навчаю поліноміальну модель із структурою

$$\hat{Y}_5 = a + b_1 X_2 + b_2 X_2^2$$

Знаходжу коефіцієнти моделі

```
In [12... x2 = df['Area']

f5 = np.polyfit(x2, y, 2)
p5 = np.poly1d(f5)
print(p5)
```

$$-1.292\text{e-}08 x^2 + 0.3826 x - 4.305\text{e}+04$$

Створюю об'єкт поліноміальної регресії та навчаю поліноміальну модель із структурою

$$\hat{Y}_6 = a + b_1 X_1 + b_2 X_2 + b_3 X_1 X_2 + b_4 X_1^2 + b_5 X_2^2$$

Знаходжу коефіцієнти моделі

```
In [13... poly3 = PolynomialFeatures(degree=2, include_bias=False)
X3_poly = poly3.fit_transform(X3)
lm6 = LinearRegression()
lm6.fit(X3_poly, Y)

print('features =', poly3.get_feature_names_out(['Population', 'Area']))
print('intercept =', lm6.intercept_)
print('coef =', lm6.coef_)
```

```
features = ['Population' 'Area' 'Population^2' 'Population Area' 'Area^2']
intercept = 55052.78023916745
coef = [-5.75800009e-16 -9.40990043e-17 -1.50841420e-12  1.03506034e-09
        -1.35507959e-09]
```

Завдання #4:

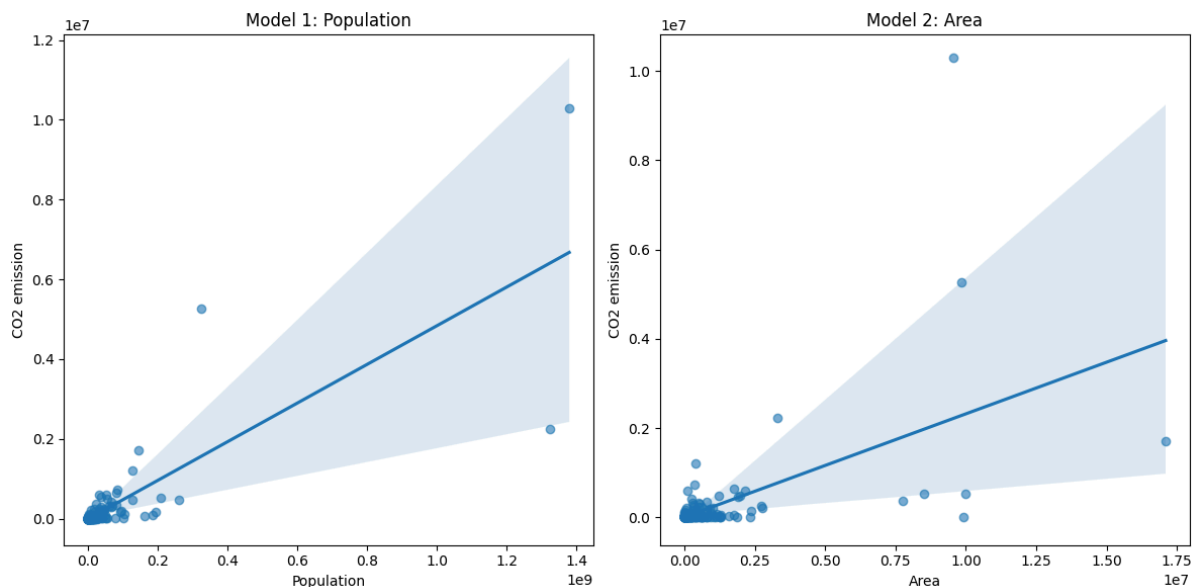
Побудуйте візуалізації для оцінки всіх моделей.

Для простої лінійної регресії чудовим способом візуалізації відповідності моделі є використання графіків регресії. Цей графік покаже комбінацію розсіяних точок даних (діаграма розсіювання, scatterplot) та підігнану лінію лінійної регресії, що проходить через дані.

Таким способом візуалізую перші моделі.

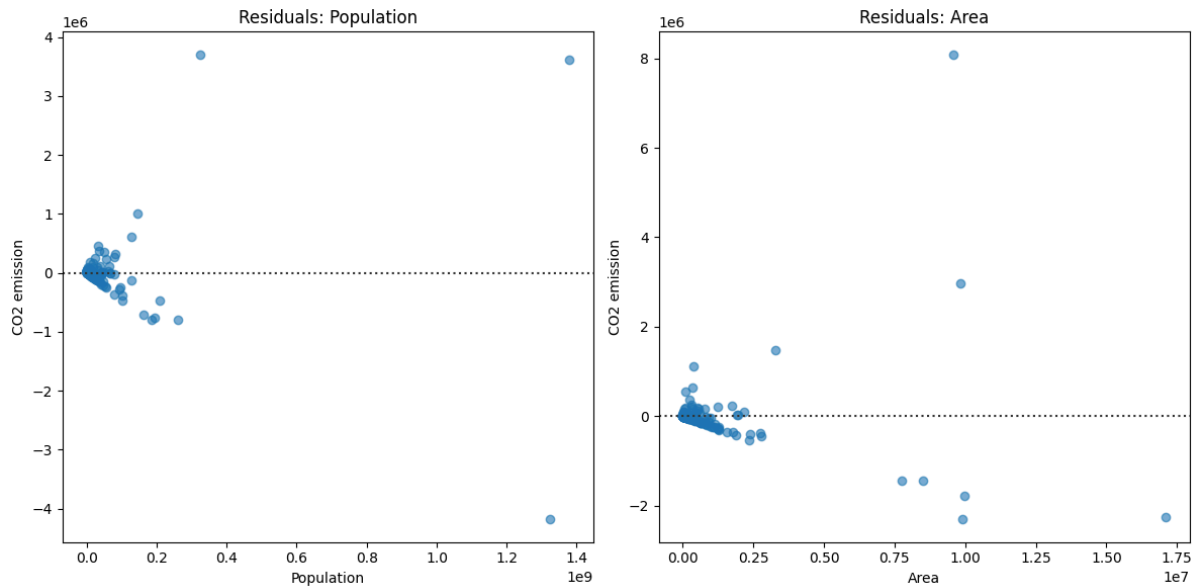
```
In [14... %matplotlib inline
width = 12
height = 6

fig, axes = plt.subplots(1, 2, figsize=(width, height))
sns.regplot(x='Population', y='CO2 emission', data=df, ax=axes[0], scatter_kws={'s': 50})
axes[0].set_title('Model 1: Population')
sns.regplot(x='Area', y='CO2 emission', data=df, ax=axes[1], scatter_kws={'s': 50})
axes[1].set_title('Model 2: Area')
plt.tight_layout()
```



Для візуалізації дисперсії даних використаю діаграми залишків - по ній зможу прийняти рішення про необхідність ускладнення моделі.

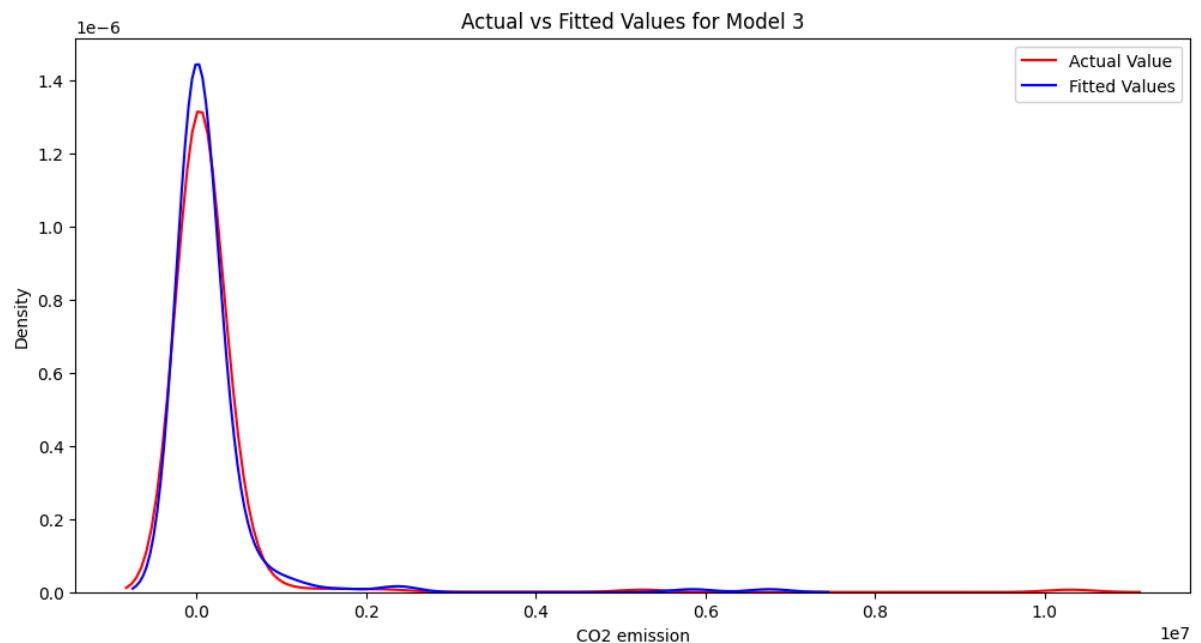
```
In [15... fig, axes = plt.subplots(1, 2, figsize=(width, height))
sns.residplot(x=df['Population'], y=df['CO2 emission'], ax=axes[0], scatter_kws={'s': 50})
axes[0].set_title('Residuals: Population')
sns.residplot(x=df['Area'], y=df['CO2 emission'], ax=axes[1], scatter_kws={'s': 50})
axes[1].set_title('Residuals: Area')
plt.tight_layout()
```



Для візуалізації моделі множинної лінійної регресії використаю діаграму розподілу.

In [16...

```
Yhat3 = lm3.predict(X3)
plt.figure(figsize=(width, height))
ax1 = sns.kdeplot(Y, color='r', label='Actual Value')
sns.kdeplot(Yhat3, color='b', label='Fitted Values', ax=ax1)
plt.title('Actual vs Fitted Values for Model 3')
plt.xlabel('CO2 emission')
plt.legend()
plt.show()
```

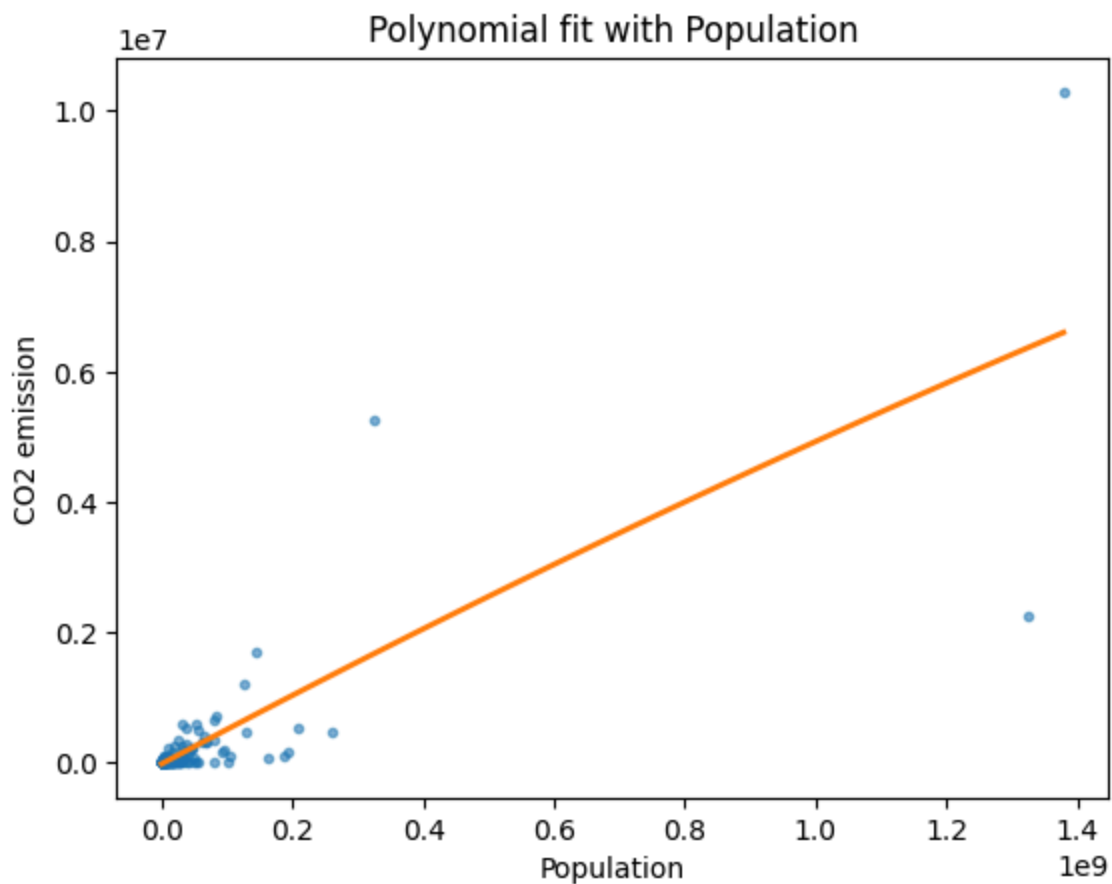


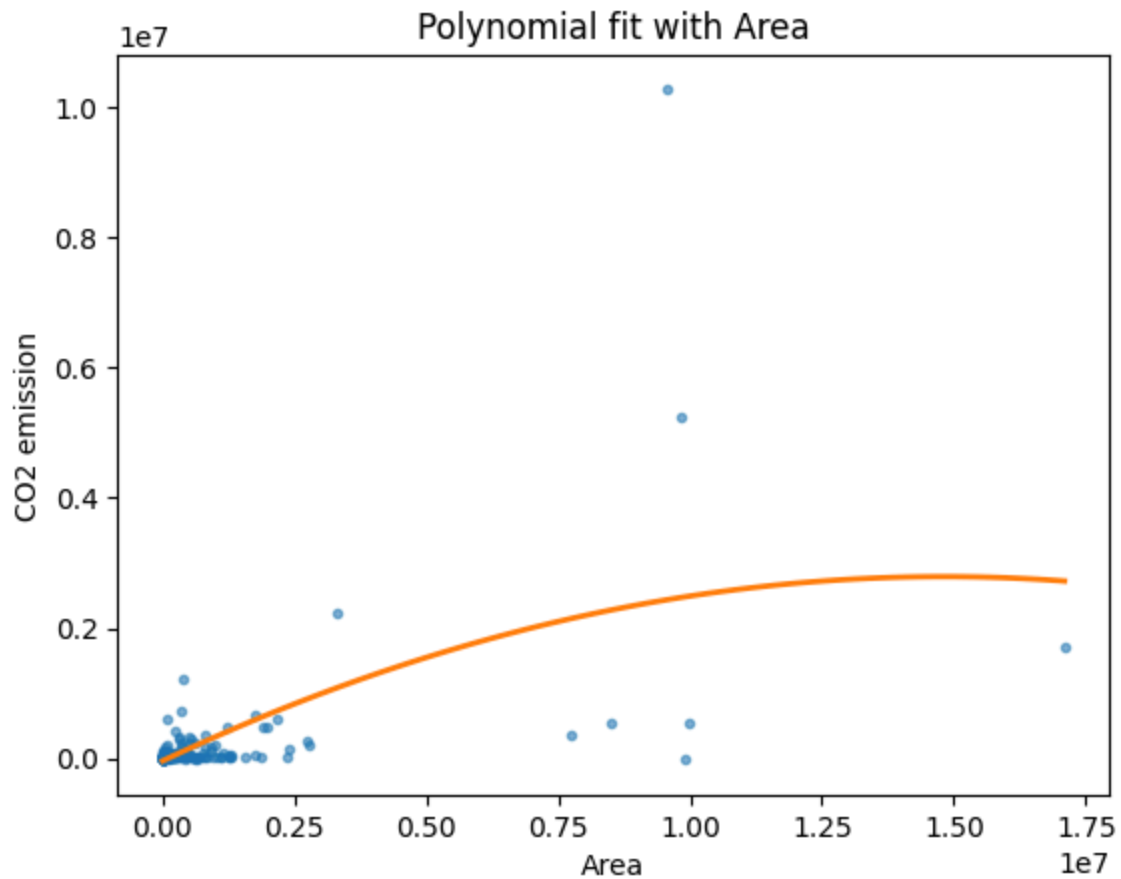
Для візуалізації поліноміальних моделей з одним предиктором використаю функцію `poly1d`. Зручно розміщувати всі моделі на одному графіку для обрання найкращої.

```
In [17... def PlotPolly(model, independent_variable, dependent_variable, name):
    x_new = np.linspace(independent_variable.min(), independent_variable.max(), 100)
    y_new = model(x_new)

    plt.plot(independent_variable, dependent_variable, '.', alpha=0.55)
    plt.plot(x_new, y_new, '-', linewidth=2)
    plt.title('Polynomial fit with ' + name)
    plt.xlabel(name)
    plt.ylabel('CO2 emission')
    plt.show()

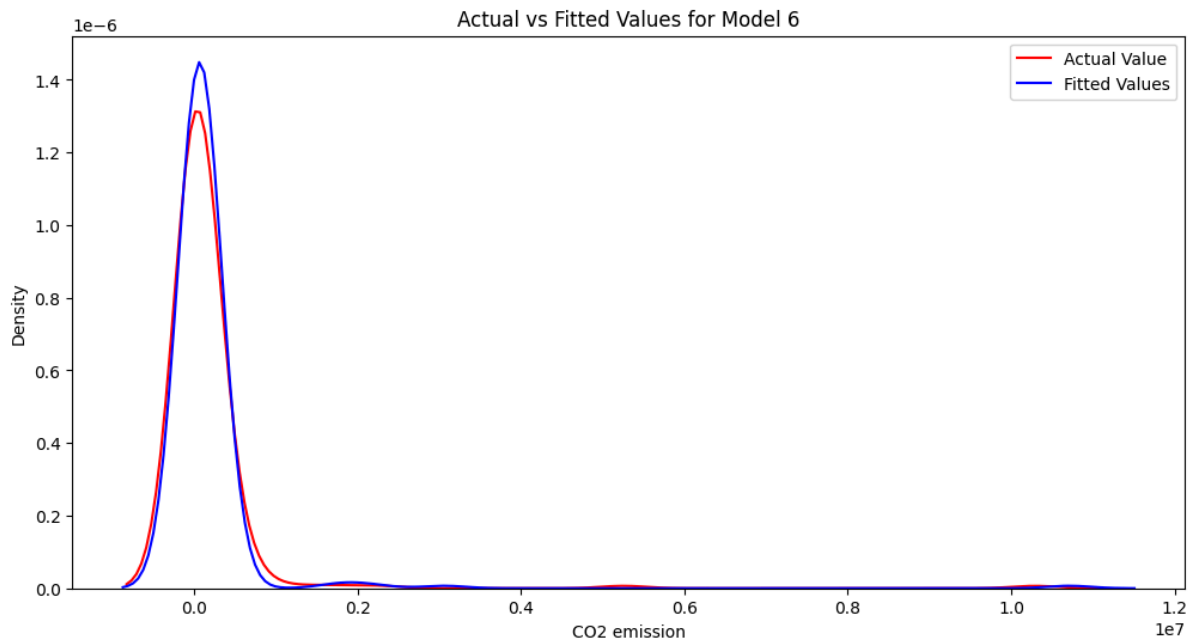
PlotPolly(p4, x1, y, 'Population')
PlotPolly(p5, x2, y, 'Area')
```





Для візуалізації поліноміальних моделей з кількома предикторами використаю діаграму розподілу або діаграму залишків

```
In [18... Yhat6 = lm6.predict(X3_poly)
plt.figure(figsize=(width, height))
ax1 = sns.kdeplot(Y, color='r', label='Actual Value')
sns.kdeplot(Yhat6, color='b', label='Fitted Values', ax=ax1)
plt.title('Actual vs Fitted Values for Model 6')
plt.xlabel('CO2 emission')
plt.legend()
plt.show()
```



Завдання #5:

Порахуйте значення R^2 та MSE для оцінки якості кожної моделі. Оберіть найкращу модель.

Значення R^2 отримую безпосередньо з моделі `.score(X, Y)`, а для розрахунку MSE спочатку формую прогнозовані значення `.predict(X)` і порівнюю їх з фактичними

Модель 1:

$$\hat{Y}_{hat1} = a + bX$$

```
In [19... Yhat1 = lm1.predict(X1)
print('R-square:', lm1.score(X1, Y))
print('MSE:', mean_squared_error(Y, Yhat1))
```

R-square: 0.6465629960775001
MSE: 231226211453.77188

Модель 2:

$$\hat{Y}_{hat2} = a + bX$$

```
In [20... Yhat2 = lm2.predict(X2)
print('R-square:', lm2.score(X2, Y))
print('MSE:', mean_squared_error(Y, Yhat2))
```

R-square: 0.30436950697780796
MSE: 455096669811.3789

Модель 3:

$$\hat{Y}_3 = a + b_1 X_1 + b_2 X_2$$

```
In [21... print('R-square:', lm3.score(X3, Y))
print('MSE:', mean_squared_error(Y, Yhat3))
```

R-square: 0.7010883020321408
MSE: 195554564783.1422

Модель 4:

$$\hat{Y}_4 = a + b_1 X_1 + b_2 X_1^2$$

```
In [22... # Імпортуємо функцію r2_score з модуля metrics, оскільки використовуємо інше ім'я
from sklearn.metrics import r2_score

Yhat4 = p4(x1)
print('R-square:', r2_score(Y, Yhat4))
print('MSE:', mean_squared_error(Y, Yhat4))
```

R-square: 0.6470807998651005
MSE: 230887452900.6083

Модель 5:

$$\hat{Y}_5 = a + b_1 X_2 + b_2 X_2^2$$

```
In [23... Yhat5 = p5(x2)
print('R-square:', r2_score(Y, Yhat5))
print('MSE:', mean_squared_error(Y, Yhat5))
```

R-square: 0.32507007411676425
MSE: 441553906429.6055

Модель 6:

$$\hat{Y}_6 = a + b_1 X_1 + b_2 X_2 + b_3 X_1 X_2 + b_4 X_1^2 + b_5 X_2^2$$

```
In [ ]: print('R-square:', r2_score(Y, Yhat6))
print('MSE:', mean_squared_error(Y, Yhat6))

# ---

model_metrics = pd.DataFrame({
    'Model': [
        '1: Linear Population',
        '2: Linear Area',
        '3: Multiple Linear',
        '4: Polynomial Population',
        '5: Polynomial Area',
```

```

        '6: Polynomial Population + Area'
    ],
    'R2': [
        lm1.score(X1, Y),
        lm2.score(X2, Y),
        lm3.score(X3, Y),
        r2_score(Y, Yhat4),
        r2_score(Y, Yhat5),
        r2_score(Y, Yhat6)
    ],
    'MSE': [
        mean_squared_error(Y, Yhat1),
        mean_squared_error(Y, Yhat2),
        mean_squared_error(Y, Yhat3),
        mean_squared_error(Y, Yhat4),
        mean_squared_error(Y, Yhat5),
        mean_squared_error(Y, Yhat6)
    ]
})
model_metrics.sort_values('R2', ascending=False)

```

R-square: 0.9257860529232583

MSE: 48552385939.04709

Out[]:

	Model	R2	MSE
5	6: Polynomial Population + Area	0.925786	4.855239e+10
2	3: Multiple Linear	0.701088	1.955546e+11
3	4: Polynomial Population	0.647081	2.308875e+11
0	1: Linear Population	0.646563	2.312262e+11
4	5: Polynomial Area	0.325070	4.415539e+11
1	2: Linear Area	0.304370	4.550967e+11

Висновок

Порівнюючи всі моделі, роблю висновок, що **модель 6 — поліноміальна модель другого порядку з ознаками Population та Area — є найкращою моделлю** для прогнозування **C02 emission** на основі нашого набору даних. Вона має найбільше значення R^2 та найменше значення MSE серед побудованих моделей.

Додаткове завдання:

Побудуйте та оцініть кілька поліноміальних моделей різних порядків.

1. Побудуйте кілька поліноміальних моделей різних порядків.

2. Побудуйте візуалізації для оцінки всіх моделей, починаючи з лінійної (зручно розмішувати всі моделі на одному графіку для обрання найкращої).
3. Порахуйте значення R^2 та MSE для оцінки якості кожної моделі (теж доцільно побудувати графік залежності R^2 або MSE від порядку поліному моделі).
4. Оберіть найкращу модель, висновок обґрунтуйте.

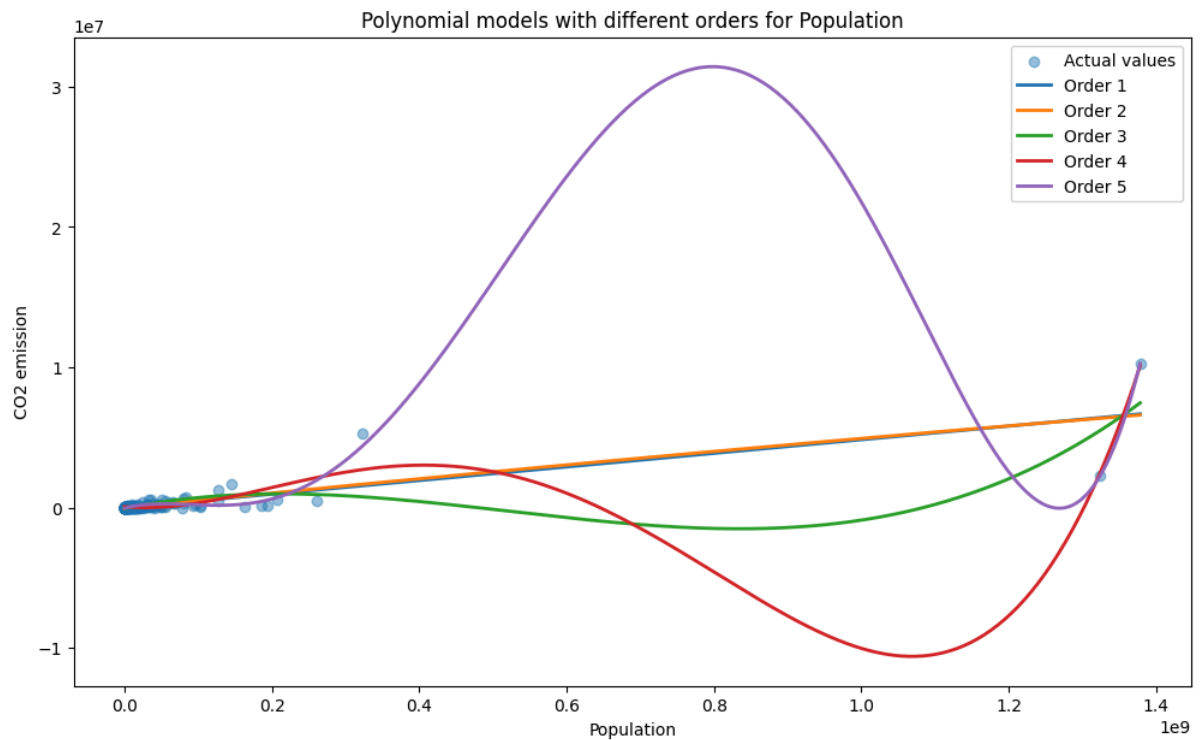
```
In [25... orders = [1, 2, 3, 4, 5]
x = df['Population']
y = Y
x_grid = np.linspace(x.min(), x.max(), 300)

extra_metrics = []
plt.figure(figsize=(12, 7))
plt.scatter(x, y, alpha=0.45, label='Actual values')

for order in orders:
    coefficients = np.polyfit(x, y, order)
    polynomial = np.poly1d(coefficients)
    y_hat = polynomial(x)
    extra_metrics.append({
        'Order': order,
        'R2': r2_score(y, y_hat),
        'MSE': mean_squared_error(y, y_hat)
    })
    plt.plot(x_grid, polynomial(x_grid), linewidth=2, label=f'Order {order}')

plt.xlabel('Population')
plt.ylabel('CO2 emission')
plt.title('Polynomial models with different orders for Population')
plt.legend()
plt.show()

pd.DataFrame(extra_metrics).sort_values('R2', ascending=False)
```



Out[25]:

	Order	R2	MSE
4	5	0.935627	4.211450e+10
3	4	0.871125	8.431262e+10
2	3	0.672907	2.139914e+11
1	2	0.647081	2.308875e+11
0	1	0.646563	2.312262e+11

Виконав студент групи ІП-45
Янов Богдан